Syefina Rifa' Maheswari
24/540626/TK/60030
ASD-C

NO
DATE

PROBLEM 1 : Algorithm Analysis

① Snippet A

```
//n >= 1
int sum = 0
for(int i = 0 ; i<n ; i++){
    for(int j = 0 ; j<i ; j++){
        sum += arr[i] + arr[j];
    }
}
```

Hint : The inner loop runs 0, 1, 2, ..., n-1 times for each value of i. use $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$

(a) Exact operation count T(n) :

Operation Table

Line	Statement	Cost / Execution	# Executions	Total Cost
1	sum = 0	1	1	1
2	i = 0	1	1	1
3	i < n	1	n+1	n+1
4	i++ (add + assign)	2	n	2n
5	j = 0	1	n	n
6	j < i	1	$\sum(i+1)$	$\frac{n(n-1)}{2}+n$
7	j++ (add + assign)	2	$\sum i$	$n(n-1)$
8	arr[i]	1	$\sum i$	$\frac{n(n-1)}{2}$
9	arr[j]	1	$\sum i$	$\frac{n(n-1)}{2}$
10	+	1	$\sum i$	$\frac{n(n-1)}{2}$
11	sum +	1	$\sum i$	$\frac{n(n-1)}{2}$
12	assignment	1	$\sum i$	$\frac{n(n-1)}{2}$

$$T(n) = 1+1+(n+1)+2n+n+\left(\frac{n(n-1)}{2}+n\right)+n(n-1)+5\frac{n(n-1)}{2}+4n+n+3$$

(b) Big-O class :

Dropping the lower-order terms (n+3) and the constant factor (3). We get : $O(n^2)$

(c) Best / Worst Case

The best and worst case cases are identical. The loops run a fixed number of times regardless of data values. No early termination.

Bantex

Dipindai dengan CamScanner

NO

DATE

② Snippet B

```
// n >= 1
int lo = 0, hi = n-1, target = 42;
bool found = false;
while (lo <= hi) {
    int mid = lo + (hi - lo)/2;
    if (arr[mid] == target)      { found = true; break; }
    else if (arr[mid] < target)  { lo = mid+1; }
    else                         { hi = mid-1; }
}
```

Hint: Each iteration halves the search space. How many times can you halve n before reaching 1?

Express T(n) in terms of $\lceil \log_2 n \rceil + 1$.

(a) Exact operation count T(n):

Let $k = \lceil \log_2 n \rceil + 1$

Operation Table

Line	Statement	cost/Execution	# Executions	Total Cost
1	lo = 0	1	1	1
2	hi = n-1	2	1	2
3	found = false	1	1	1
4	lo <= hi	1	k+1	k+1
5	hi - lo	1	k	k
6	/2	1	k	k
7	lo +	1	k	k
8	assign mid	1	k	k
9	arr[mid]	1	k	k
10	compare ==	1	k	k
11	arr[mid]	1	k	k
12	compare <	1	k	k
13	update (add + assign)	2	k	2k

$T(n) = 1+2+1+(k+1)+k+k+k+k+k+k+k+k+k+2k = 11k+6 \rightarrow k = \lceil \log_2 n \rceil + 1$

$= 11(\lceil \log_2 n \rceil + 1) + 6 = 11\lceil \log_2 n \rceil + 17$

Bantex

Dipindai dengan CamScanner

Title :

Date:

(b) Big-O class :

Dropping the lower-order terms and constant factor : $O(\log n)$

(c) Best / Worst case.

The best and worst cases are different.

→ Best case : target found immediatly

$k = 1 \rightarrow T(n) = \text{constant} \rightarrow O(1)$

→ Worst case : search continuous untill space size 1

$k = \lceil \log_2 n \rceil + 1 \rightarrow O(\log n)$

③ Snippet C

```
//n >= 1
int count = 0
for(int i = 0 ; i < n ; i++){
    for(int j = i+1 ; j < n ; j++){
        for(int k = 0 ; k < n ; k += 2){ //note : k += 2
            if(arr[i] != arr[j] && arr[k] > 0)
                count++;
        }
    }
}
```

Hint : The innermost loops runs $\lceil n/2 \rceil$ times regardless of i and j. The middle loop runs a total of $\frac{n(n-1)}{2}$ times (summed over all i).

(a) Exact operation count $T(n)$:

Line	Statement	Cost	# Executions	Total Cost
1	$i = 0$	1	1	1
2	$i < n$	1	1	1
3	$i++$	2	n	$2n$
4	$j = i+1$	2	n	$2n$
5	$j < n$	1	$\frac{n(n-1)}{2} + n$	$\frac{n^2+n}{2}$
6	$j++$	2	$\frac{n(n-1)}{2}$	$\frac{n^2-n}{2}$
7	$k = 0$	1	$\frac{n(n-1)}{2}$	$\frac{n^2-n}{2}$
8	$k < n$	1	$\left(\frac{n}{2}+1\right)\left(\frac{n(n-1)}{2}\right)$	$\frac{n^3+n^2-2n}{4}$
9	$k += 2$	2	$\frac{n}{2}\left(\frac{n(n-1)}{2}\right)$	$\frac{n^3-n^2}{2}$
10	condition (6 ops)	6	$\frac{n}{2}\left(\frac{n(n-1)}{2}\right)$	$\frac{3n^3-3n^2}{2}$
11	count++	2	$\frac{n}{2}\left(\frac{n(n-1)}{2}\right)$	$\frac{n^3-n^2}{2}$

$$T(n) = 1+1+2n+2n+\left(\frac{n^2+n}{2}\right)+\left(\frac{n^2-n}{2}\right)+\left(\frac{n^2-n}{2}\right)+\left(\frac{n^3+n^2-2n}{4}\right)+\left(\frac{n^3-n^2}{2}\right)+\left(\frac{3n^3-3n^2}{2}\right)+\left(\frac{n^3-n^2}{2}\right)$$

$$= \frac{11}{4}n^3 - \frac{3}{4}n^2 + 3n + 2$$

Dipindai dengan CamScanner

Title :
Date:

(b) Big-O Class

Dropping the constant and lower-order terms : $O(n^3)$

(c) Best / Worst case

The best and worst cases are different.

→ Worst case : condition always true → increment always executed

→ Best case : condition always false → increment never executed

However, the loop structure is identical in both cases. only constant factor differs.

so. both best and worst case are : $O(n^3)$

PROBLEM 3 : Linear Search vs Binary search

Ⓑ Complexity Comparison Table

Operation	Best Case	Worst Case	Average Case
Linear search (unsorted)	O(1) - Elemen target ditemukan pada posisi pertama sehingga hanya satu perbandingan diperlukan	O(n) - Target berada di posisi terakhir atau tidak ada dalam array sehingga seluruh n elemen diperiksa	O(n) - secara rata-rata diperlukan sekitar $\frac{n}{2}$ perbandingan yang tetap proposional terhadap n.
Linear search (sorted, early exit)	O(1) - Target ditemukan pada elemen pertama	O(n) - Target lebih besar dari seluruh elemen sehingga semua elemen tetap diperiksa	O(n) - Walaupun pencarian dapat berhenti lebih awal jika elemen terlompati, jumlah pemeriksaan rata-rata tetap bertumbuh linear terhadap n
Binary search (sorted)	O(1) - Target tepat berada di tengah pada pemeriksaan pertama	O(log n) - setiap iterasi membagi ruang pencarian menjadi setengah hingga tersisa satu elemen	O(log n) - Rata-rata jumlah langkah tetap sebanding dengan tinggi pohon biner pencarian, yaitu $\log_2 n$

→ Linear search worst case $\approx n$

Binary search worst case $\approx \log_2 n$

→ Binary search lebih efisien jika $\log_2 n < n$

→ uji nilai kecil n :

n	$\log_2 n$	Perbandingan
1	0	$0 < 1$
2	1	$1 < 2$
3	1.58	$1.58 < 3$
4	2	$2 < 4$

Secara matematis :

Untuk semua $n \geq 2$, berlaku : $\log_2 n < n$

Karena pertumbuhan logaritmik jauh lebih lambat dibanding pertumbuhan linear.

→

Dipindai dengan CamScanner

Title:
Date:

→ Binary search menjadi lebih efisien dari linear search dalam worst case mulai dari $n \geq 2$ karena untuk semua $n \geq 2$ berlaku $O(n) < O(n)$

PROBLEM 4 : Amortized Analysis of pushBack

(A) Trace

→ kapasitas awal = 2

→ Jika size == capacity, maka capacity = 2 × capacity

→ cost : Normal append = 1

Resize = 1 + old size

→ Awal : size = 0, capacity = 2

Call ※	Value Pushed	size after	capacity after	cost
1	10	1	2	1
2	20	2	2	1
3	30	3	4	1+2=3
4	40	4	4	1
5	50	5	8	1+4=5
6	60	6	8	1
7	70	7	8	1
8	80	8	8	1
9	90	9	16	1+8=9
10	100	10	16	1
11	110	11	16	1
12	120	12	16	1

(B) Total Cost

1) $T_{12} = 1+1+3+1+5+1+1+1+9+1+1+1 = 26$

2) Average cost $= \frac{T_{12}}{12} = \frac{26}{12} \approx 2.167$

3) → Resizes terjadi saat kapasitas 2 4 8 16 32 .....

→ setiap resize menyalin sebanyak kapasitas lama : $2+4+8+16+\ldots+k$

→ deret geometri : $\sum_{i=1}^{k} 2i = 2^{k+1} - 2$

→ Karena $2^k \leq n$, maka Total copy work $< 2n$

→ Total cost keseluruhan : $T(n) = n + O(n)$

→ Amortised Cost : $\frac{T(n)}{n} = O(1)$

→ Kesimpulan : strategi doubling menghasilkan amortised cost konstan.

⟶

Dipindai dengan CamScanner

Title :
Date:

ⓒ Growth factor comparison.

- Aturan baru : capacity = old capacity + 4
- kapasitas menjadi 2 6 10 14 18 ....

1) Trace 12 calls

Call #	Value	size after	capacity after	Cost
1	10	1	2	1
2	20	2	2	1
3	30	3	6	1+2=3
4	40	4	6	1
5	50	5	6	1
6	60	6	6	1
7	70	7	10	1+6=7
8	80	8	10	1
9	90	9	10	1
10	100	10	10	1
11	110	11	14	1+10=11
12	120	12	14	1

$T_{12}(new) = 1+1+3+1+1+1+7+1+1+1+11+1 = 30$

2) Comparison

strategy	Total cost (12 calls)
Doubling	26
+4 Fixed	30

Untuk n kecil perbedaannya kecil, tetapi untuk n besar perbedaannya signifikan.

3) General Analysis

↳ Doubling strategy

Copy work : 2+4+8+16+...

Deret Geometri : $< 2n$

$T(n) = O(n)$

Amortised : $O(1)$

↳ Fixed Increment strategy

- Resizes terjadi di 2, 6, 10, 14, ...

Copy work : 2+6+10+14+.... (Deret Aritmetika)

- Jika jumlah resize $\approx \frac{n}{4}$, maka :

$$\text{Total copy work} \approx \sum_{i=0}^{n/4} (2+4$$

- Rumus deret aritmetika : $= \frac{m}{2}$ (first + last)

karena $m \propto n$ dan last term $\propto n$, maka $T(n) = O(n^2)$

- Amortised cost : $\frac{O(n^2)}{n} = O(n)$

Dipindai dengan CamScanner

Title :

Date:

↳ Final conclusion

strategy	Total Cost	Amortised Cost
Doubling	$O(n)$	$O(1)$
Fixed +4	$O(n^2)$	$O(n)$

strategi doubling jauh lebih efisien secara asimtotik karena total cost tetap linear terhadap n, sedangkan strategi penambahan tetap menghasilkan total cost kuadratik.

Dipindai dengan CamScanner